vpn_failclosed_test.sh

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z

Standing §11.4.68/§11.4.69/§11.4.108/§11.4.115 fail-closed **safety** guard for the VPN-aware `dynamic` routing profile. It proves the security-critical contract: **when the VPN tunnel for a request is DOWN, the proxy serves the branded fail-closed response (`503` + `ERR_TUNNEL_DOWN`) and MUST NOT leak the request to the open internet.**

Overview

Field	Value
Path	`tests/dynamic/vpn_failclosed_test.sh`
Sources	`tests/lib/evidence.sh` (`assert_graceful_503`, `ab_pass_with_evidence`, `ab_skip_with_reason`)
Under test	`docker-compose.dynamic.yml` `dynamic` profile — acl-helper → Redis → Squid `deny_info 503:ERR_TUNNEL_DOWN`
Kind	Live fail-closed guard (GREEN) + fixture-driven self-validation (RED)
Live-boot owner	The **conductor** (`./start --dynamic`), NOT this script (§11.4.119 single owner)
Anti-bluff anchors	§11.4.68, §11.4.69, §11.4.108, §11.4.115, §11.4.107(10), §11.4.3, §11.4.21, §11.4.50, §1.1
Exit codes	`0` = PASS / valid-SKIP, `1` = FAIL, `2` = invalid-SKIP

How the tunnel-down state is set (without a real VPN)

The acl-helper (`control-plane/cmd/acl-helper/main.go`) is data-driven from Redis via the pure verdict in `control-plane/internal/aclhelper/decide.go:47-70`:

- It answers `OK tag=<tunnel>` **only** when `route:<host>` exists **AND** `vpn:status:<tunnel>.state == "up"`.

- **Every other case** — no route, empty tunnel, transport error, and an explicit `state: "down"` / `"unknown"` / missing / stale key — returns `ERR`.

Squid turns `ERR` into `deny_info 503:ERR_TUNNEL_DOWN` (`control-plane/internal/routing/routing.go:294-297` rendered into the include; `config/squid/squid.dynamic.conf:75-104`), which serves the baked branded page `config/squid/errors/ERR_TUNNEL_DOWN`.

So the test forces the tunnel-down fail-closed path **deterministically with no WireGuard credentials and no gluetun egress** by writing, into `proxy-redis`:

- `route:<host>` = `{"target":"<host>","tunnel":"<profile>",...}` — so the helper reaches the *status* check (exercises the *tunnel-down* path, not merely *no-route*).
- `vpn:status:<profile>` = `{"profile":"<profile>","state":"down",...}` — the tunnel is DOWN (`control-plane/internal/redis/client.go:82-101` `evaluateStatus`; keys/JSON shape `control-plane/internal/vpn/vpn.go:21-41`).

`healthd` also publishes the same profile-set DOWN on its own loop when gluetun has no real egress, so nothing can race the status back to "up".

Prerequisites

- The **conductor** has booted the dynamic stack and declared it up: `./start --dynamic` then export `HELIX_DYNAMIC_STACK=1` (§11.4.119 — this script never boots/starts/stops/builds anything).
- A container runtime (`podman` preferred, else `docker`) to `exec redis-cli` into `proxy-redis` and read the Squid PID — **exec, not a start/stop workflow** (mirrors `lib/container-runtime.sh`'s own `podman exec` / `podman ps` use). Fully overridable via `HELIX_FAILCLOSED_REDIS_CLI` (§11.4.28 config injection).
- `curl`, `awk`, `grep`, POSIX `sh`.
- **No** WireGuard credentials are required — the fail-closed half is autonomous.

Usage examples

```
# GREEN standing safety guard (conductor has booted the dynamic
        stack):
HELIX_DYNAMIC_STACK=1 bash tests/dynamic/vpn_failclosed_test.sh

# RED self-validation (§11.4.115) — proves the branded-503
        assertion catches a
# leak; needs NO live stack:
```

```
RED_MODE=1 bash tests/dynamic/vpn_failclosed_test.sh

# Under the mandated host caps (self-applied when nice/ionice are
        present):
HELIX_DYNAMIC_STACK=1 GOMAXPROCS=2 nice -n 19 ionice -c 3 \
    bash tests/dynamic/vpn_failclosed_test.sh
```

Exact conductor boot + run sequence:

```
./start --dynamic                                    # boots the
        dynamic profile, binds :34128
HELIX_DYNAMIC_STACK=1 bash tests/dynamic/vpn_failclosed_test.sh
```

What it asserts (GREEN)

1. **Availability gate** — `HELIX_DYNAMIC_STACK=1` declared, `proxy-redis` reachable (`redis-cli ping` → `PONG`), and the proxy reachable; otherwise an honest §11.4.3 SKIP (`topology_unsupported` / `network_unreachable_external`), never a fake PASS.
2. **Inject tunnel-down** — write `route:<host>` + `vpn:status:<profile>=down`, then re-`GET` and confirm `"state":"down"` took (captured in `redis_down_state.txt`).
3. **Branded fail-closed 503** — `HELIX_FAILCLOSED_ITER` (default 3, §11.4.50) proxied requests to the target; every one must be HTTP `503` **with `ERR_TUNNEL_DOWN` in the body** (positive branded evidence, not the origin's content).
4. **No leak** — Squid's own `access.log` must show `TCP_DENIED/503` for the host and **no** upstream-forward line (`HIER_DIRECT` / `*_PARENT`); any `2xx`/`3xx` through the proxy or any upstream forward is a **hard FAIL** (the exact safety violation).
5. **Runtime signature (§11.4.108)** — canonical `assert_graceful_503` corroborates the branded body **and** that the Squid PID is unchanged (no crash/restart).
6. **Egress half (B)** — proving traffic actually exits *via the tunnel* is **operator-gated on gluetun WireGuard credentials (§11.4.21)** and is emitted as an honest `operator_attended` SKIP — **not attempted** here.

Verdict matrix (first match wins):

Observation	Verdict
Any `2xx`/`3xx` through proxy, or an upstream-forward log line	**FAIL** (LEAK)
Every iter `503` + `ERR_TUNNEL_DOWN` + no leak + PID unchanged	**PASS** (+ operator-attended egress SKIP)
No leak, but branded path inactive (external_acl not rendered)	SKIP `feature_disabled_by_config`

Observation	Verdict
Proxy unreachable	SKIP `network_unreachable_external`

RED self-validation (§11.4.115 / §11.4.107(10))

`RED_MODE=1` feeds a **golden-BAD** fixture — a fabricated `200` "leak" body carrying the origin's content and **no** `ERR_TUNNEL_DOWN` — into the *same* canonical `assert_graceful_503` path (via the documented `EVIDENCE_503_*` unit seams) and asserts it **FAILs**. A branded-503 assertion that PASSed a `200` leak would be a bluff gate. Captured: `qa-results/dynamic/vpn_failclosed/<run-id>/red_baseline.txt` (`assert_graceful_503 rc=1 — HTTP code 200 != 503`).

Edge cases

- **Redis unreachable** → SKIP `topology_unsupported` (the down-state write is the whole test; a fabricated fail-closed PASS is forbidden).
- **`vpn:status` write did not take** → hard FAIL (the precondition is unproven).
- **Branded include not rendered** (compiler lane pending) → the request still fails closed (no leak) but the `ERR_TUNNEL_DOWN` page is absent → honest `feature_disabled_by_config` SKIP, never a fail-open PASS (§11.4.68) and never a false leak-FAIL.
- **Origin genuinely offline** → the branded-**body** assertion still discriminates a real fail-closed 503 (branded) from an upstream-connect failure (Squid's own error page, no `ERR_TUNNEL_DOWN`).

Internal behaviour

1. Self-re-exec under `nice -n 19 ionice -c 3` + `GOMAXPROCS=2` (§12.6), `set -u`, source `tests/lib/evidence.sh`.
2. `RED_MODE=1` → fixture self-validation (no stack) and exit.
3. `RED_MODE=0` → availability gate → inject down-state → N proxied requests → access.log no-leak parse → canonical `assert_graceful_503` → aggregate verdict.
4. `trap ... EXIT` deletes **only** the test's own `route:<host>` + `vpn:status:<profile>` keys (§11.4.14 — leaves the stack quiescent).
5. All artefacts under the gitignored `qa-results/dynamic/vpn_failclosed/<run-id>/`.

§1.1 paired-mutation proof

The RED mode **is** the paired mutation: it drives the GREEN assertion against a known leak and requires it to FAIL. Mutating `assert_graceful_503` in `tests/lib/evidence.sh` to accept a non-503 (e.g. remove the `code != 503` arm) flips `RED_MODE=1` to a `FAIL` here (the assertion no longer catches the leak), proving this guard is not a bluff gate.

Related scripts

- `tests/lib/evidence.sh` — canonical `assert_graceful_503` / `assert_no_leak` / `ab_pass_with_evidence` / `ab_skip_with_reason`.
- `tests/dynamic/suites/chaos_suite.sh` — kill-tunnel / drop-net / corrupt-redis fail-closed chaos (sibling; delegates fault injection to `HELIX_CHAOS_*` cmds).
- `tests/dynamic/analyzers/graceful_503_analyzer.sh` — golden-good/golden-bad self-validated branded-503 analyzer.
- `tests/observability/metrics_scrape_test.sh` — the `helix_proxy_tunnel_down_responses_total` counter (the metrics view of this same fail-closed event).

Last verified

2026-07-01 — `sh -n` + `bash -n` clean; no-stack run emits the honest `topology_unsupported` SKIP; `RED_MODE=1` PASS with captured evidence (`assert_graceful_503 rc=1`, `HTTP code 200 != 503`). The GREEN live run is owed to the conductor's `./start --dynamic` boot (§11.4.119).